

Boxed: A Docker-Based Code-Execution Substrate for Autonomous Code-Generating Agents

Akshay Kumar
Independent Researcher
United States
akumar8@mt.iitr.ac.in

Abstract—Autonomous large language model (LLM) agents increasingly write, execute, and iterate on code inside their reasoning loop, which makes the sandbox, not the model, the hot path of an agent task. Operators today choose among three unsatisfying options: roll-your-own Docker, which is fast to prototype but routinely ships without seccomp profiles, egress filtering, read-only root filesystems, or process caps; proprietary SaaS sandboxes, which are hardened but vendor-locked or unsuitable for an operator’s deployment requirements; and research virtual machine monitors, which provide the right isolation primitives but target a boot-once serverless invocation model rather than the agent’s create/execute/destroy cycle. We present Boxed, an open-source sandbox substrate co-designed for agentic code execution. Boxed contributes a deliberately narrow four-method Go driver interface that makes the isolation backend an extension point; a 1.32 MiB stripped Rust in-sandbox agent that streams stdout, stderr, and on-disk artifacts over JSON-RPC 2.0 as they are produced; and a stateless control plane that an operator can self-host. The implementation currently provides one Docker backend; Firecracker and WebAssembly remain stubs. The Docker backend defaults to a read-only root filesystem, no Linux capabilities, no-new-privileges, a PID limit, and no network connectivity. Historical benchmark traces are included for reproducibility, but they predate these hardening changes and an exit-code reporting correction; they are not claims about the revised implementation. Boxed is MIT-licensed at <https://github.com/akshayaggarwal99/boxed>.

Index Terms—sandboxing, code execution isolation, large language models, autonomous agents, containers, Docker, JSON-RPC

I. Introduction

The 2024–2026 wave of autonomous code-generating agents, from Yang et al.’s SWE-agent [1] and AutoGPT-style harnesses through commercial products such as Cursor, Devin, and Claude Code, has changed what “running untrusted code” means in practice. Executable code has been argued to be a more effective action space for an agent than text or JSON [2], and a modern agent on that pattern emits dozens of short-lived programs per task: it generates a snippet, executes it, inspects stdout, reads back a generated file, and iterates. Unlike batch compilation or single-shot execution, the sandbox is not a post-hoc verification step but a component of the agent’s decision loop. The hot path of the agent is the sandbox, not the model.

Today’s options for hosting that hot path fall into three buckets, each unsatisfying for an operator who cares about cost, latency, and control over deployment simultaneously.

Roll-your-own Docker is the modal choice. It is fast to prototype but routinely shipped without seccomp profiles, network egress filters, read-only root filesystems, or process and memory caps, leaving the host one rm -rf / away from compromise [3]. Measurement studies of deployed containers report the same pattern at scale [4], and cgroup-based resource isolation has itself been shown to be evadable [5].

Proprietary SaaS sandboxes (E2B [6], Modal Sandbox [7], Cloudflare Sandbox [8]) ship hardened isolation but require the operator to transmit every byte of generated code, execution output, and intermediate artifact to a third party, which can conflict with an operator’s deployment and data-handling requirements.

Research VMMs such as Agache et al.’s Firecracker [9] provide the right primitives but are designed for AWS Lambda’s invocation model (boot once, run an event loop, amortize startup across many requests) rather than the agent’s create/execute/destroy cycle, and they ship without an in-guest agent that can stream artifacts back to the caller.

Agentic workloads call for a sandbox substrate that treats the LLM client as a first-class citizen: pluggable isolation backends so one control plane drives Docker on a laptop and Firecracker in production, an in-sandbox agent that streams standard output and on-disk artifacts as they appear, and a self-hosted deployment story in which the operator controls the control plane and its workload data.

A. Problem Statement

An autonomous coding agent executes untrusted, machine-generated code many times within a single task, so the sandbox sits on the hot path rather than at the edge of the system. Three properties are required at once. The isolation boundary must be strong enough to contain code the operator never reviewed. The per-execution latency must be low enough that the sandbox does not dominate the agent’s loop. The deployment must leave the operator in control of the keys and the data, because agent workloads routinely carry proprietary source and customer records into the sandbox.

No existing substrate satisfies all three. Proprietary sandbox services meet the first two but place a third party inside the trust boundary and expose no control over the isolation backend. Self-hosted container runtimes leave the operator sovereign but couple the control plane to one isolation technology, so strengthening the boundary later means rewriting the orchestration layer. The gap addressed here is a substrate that treats the isolation backend as a deployment-time choice while keeping the control plane, the keys, and the data under the operator’s control.

B. Objectives

- Define an isolation interface narrow enough that container runtimes, microVMs, and WebAssembly sandboxes are interchangeable behind it, so that the security boundary can be strengthened without changing the control plane.
- Deliver execution output incrementally, so that an agent can react to partial results rather than waiting for process exit, and return files produced during execution as structured events rather than as an out-of-band artifact.
- Support self-hosted deployment without requiring a Boxed-operated service to receive workload data.
- Quantify the cost and the limits of the resulting design on commodity hardware, including the isolation failures the current configuration does not prevent.

C. Contributions

- A driver interface that abstracts container runtimes, microVMs, and WebAssembly sandboxes behind four Go methods, making the isolation backend a deployment-time configuration choice rather than a compile-time dependency. The current release ships a Docker driver together with stubs for Firecracker and WebAssembly.
- A Rust in-sandbox agent that streams stdout and stderr chunks as discrete JSON-RPC events instead of buffering until process exit, and mirrors files written under a watched directory back to the caller as structured artifact events.
- A self-hosted control plane with an operator-configured API key. It is a single-instance authentication mechanism, not a credential-management or multi-tenant system.
- An open benchmark harness covering cold-start latency, throughput under concurrency, per-sandbox idle overhead, a twelve-vector adversarial escape probe with full per-vector results, and an end-to-end agent-driven code-generation trace. Raw traces ship with the source.
- A benchmark harness and a pre-hardening negative-results baseline. The paper identifies the limits of that baseline and specifies the reruns required before publication.

II. Background and Related Work

A. Container-based isolation

Linux containers built on namespaces, cgroups, and seccomp underpin nearly every modern development sandbox. They are cheap to start, roughly 50–200ms when images are warm, which is why they dominate workloads that create and destroy execution environments frequently. That speed comes at a security cost: containers share the host kernel, exposing a substantially larger attack surface than hardware-isolated alternatives. The consequence is not merely theoretical. CVE-2019-5736 [3] is the canonical reminder that a single bug in runc can turn a container into a host. Hardened container runtimes such as gVisor [10], whose syscall-interception overhead Young et al. [11] quantify, interpose a userspace kernel between the workload and the host, shrinking the trusted computing base at the cost of a 2–5 $\times$ syscall overhead. For an agent executing hundreds of short-lived programs per task, that multiplicative cost must be weighed against the reduced risk of kernel-level compromise.

B. Lightweight virtual machines

Agache et al. [9] demonstrated that a stripped-down KVM VMM can boot a guest kernel in ~ 125 ms with <5 MiB of memory overhead per VM, and now powers AWS Lambda. Kata Containers [12] and Manco et al.’s My VM is Lighter (and Safer) than Your Container [13] pursue the same hypervisor-as-isolation thesis from complementary angles. These systems optimize boot cost and steady-state memory overhead under a serverless assumption: boot once, run an event loop, amortize startup across many invocations. That assumption diverges from the access pattern of an autonomous coding agent, which creates an ephemeral sandbox, runs one short-lived program, extracts outputs, and destroys the environment, hundreds of times per task. It is the cumulative lifecycle cost, not cold-start latency in isolation, that determines whether a substrate is viable for agentic workloads, and the lightweight-VM literature offers comparatively little guidance on that regime.

C. Unikernels and WebAssembly

Madhavapeddy et al. [14] push isolation further by compiling the application together with a library operating system into a single boot image, eliminating the general-purpose kernel layer and minimizing the trusted computing base, at the cost of requiring application-specific compilation, which precludes executing arbitrary dynamically generated binaries. WebAssembly [15] and WASI [16] reframe the problem at the language level: the guest is portable bytecode with explicit capability-based imports, and the “sandbox” is a runtime embedded in the host process. Microsecond-scale instantiation is exactly what a create-heavy agent workload wants. The obstacle is workload compatibility: agents emit unmodified Python scripts, shell commands, and native binaries that

assume a POSIX environment with filesystem access, process spawning, and dynamic linking. Until the WASI toolchain supports unmodified language runtimes and arbitrary native executables, Wasm remains promising but impractical as a general-purpose agent sandbox.

D. Agent-oriented sandbox platforms

A recent generation of products targets LLM agents specifically. E2B [6] runs Firecracker microVMs behind a SaaS API and reports ~ 150 ms cold start; Modal Sandbox [7] layers gVisor over Firecracker for defence in depth; Daytona [17] ships an AGPL Docker-backed self-hosted development environment; Cloudflare Sandbox [8] reuses V8 isolates for ~ 5 ms cold start at the cost of restricting execution to JavaScript and Wasm. Cold-start figures for these platforms are vendor claims measured under undisclosed conditions and are not directly comparable to our end-to-end lifecycle measurement; we reproduce them in Table I as reported, not as validated. Two limitations are shared across the category. First, none exposes a pluggable isolation backend: the choice of Docker, Firecracker, or V8 is fixed at the platform level and cannot be re-targeted from a laptop to a production cluster without migrating systems. Second, with the exception of Daytona, all operate primarily as hosted services, which can be unsuitable for deployments that require the operator to retain control of workload data. Table I positions Boxed within this landscape.

E. Evaluating LLM-generated code execution

Chen et al. [18] established single-function code generation as a standard benchmark, measuring whether a model can synthesize a correct implementation from a natural-language specification. Yang et al. [1] extended this to agent-mediated software engineering, where the agent must run the code it writes and refine it, not merely emit it. Our agent-trace evaluation (Section VI) follows the latter regime: the model emits a candidate Python function, Boxed executes it alongside a self-checking assertion, and the harness records pass or fail. By decomposing end-to-end wall-clock into model inference versus sandbox lifecycle, we quantify the part of agent latency that a substrate can actually influence.

III. Threat Model

The threat model assumes an adversary with full control over the code submitted to a sandbox: they may craft arbitrary Linux binaries, issue any syscall sequence, and attempt network egress to any destination. The adversary does not control the control plane, the driver implementation, or the host kernel; these remain trusted. Fig. 1 shows the resulting trust boundary.

The design aims to prevent four classes of attack.

T1 (Host filesystem access. Read or write access to the host filesystem outside the sandbox’s ephemeral root, whether through path manipulation, symbolic links, or direct mount operations.

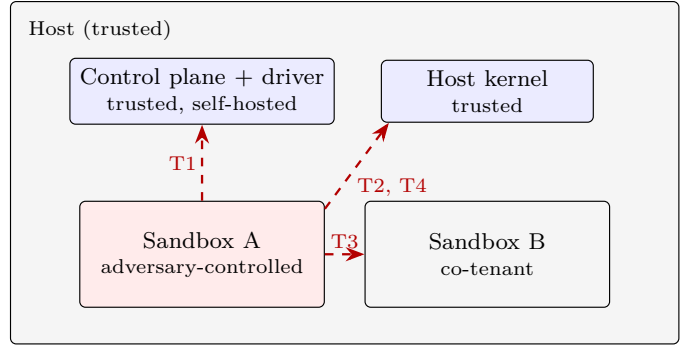


Fig. 1. Trust boundary. Adversary-controlled code executes inside a sandbox; the control plane, driver, and host kernel are trusted. Dashed arrows are the four threat classes the substrate must resist. Side channels between co-tenants on shared hardware are out of scope.

T2) Namespace escalation. Escape from the allocated network or process namespace into the host’s network stack or process tree, which would let the adversary observe or manipulate unrelated system activity.

T3, Co-tenant exfiltration. Access to credentials, artifacts, or data belonging to a concurrent sandbox on the same host.

T4, Resource exhaustion. Denial of service against the host through fork bombs, memory exhaustion, or other unbounded resource consumption.

Side-channel attacks exploiting shared hardware (cache timing, speculative execution) are explicitly out of scope. Mitigating them requires per-customer hardware partitioning, a deployment decision that belongs to the operator rather than a property the substrate can enforce.

IV. Design

Boxed is structured around three components and one narrow interface (Fig. 2): a stateless control plane, a pluggable driver layer, and an in-sandbox agent. The design decouples orchestration from the isolation mechanism so that the same control plane drives Docker containers on a development laptop and Firecracker microVMs in production without modification.

A. Driver Interface

The central abstraction is a single Go interface whose sandbox-lifecycle subset is deliberately narrow, so that future drivers (Firecracker, Wasm, Kata) can be added without touching the control plane. The full interface also exposes file-injection and listing helpers used by the SDK, but four lifecycle methods constitute the minimal contract every backend must satisfy:

```
type Driver interface {
    Create(ctx context.Context,
        cfg SandboxConfig) (string, error)
    Start(ctx context.Context, id string) error
    Stop(ctx context.Context, id string) error
    Connect(ctx context.Context,
        id string) (io.ReadWriteCloser, error)
```

TABLE I

Agent-oriented sandbox runtimes (data as of 2026-05). Cold-start figures for systems other than Boxed are vendor claims measured under undisclosed conditions and are reproduced as reported, not validated by us; the Boxed figure is the end-to-end create+exec+destroy median of Section VI.

System	Isolation	Cold start	Pluggable drivers	BYOK / self-host	License	First-class artifacts
Boxed (this work)	Docker → FC* / Wasm*	303 ms (measured)	yes	yes	MIT	yes
E2B [6]	Firecracker	~150 ms (claim)	no	limited	Apache-2.0	yes
Modal Sandbox [7]	gVisor + Firecracker	~1.5 s (claim)	no	no	proprietary	yes
Daytona [17]	Docker	not reported	no	yes	AGPL	no
Cloudflare Sandbox [8]	V8 isolates	~5 ms (claim)	limited	no	proprietary	limited

*Stubbed; Docker is the only production-ready driver in the current Boxed release.

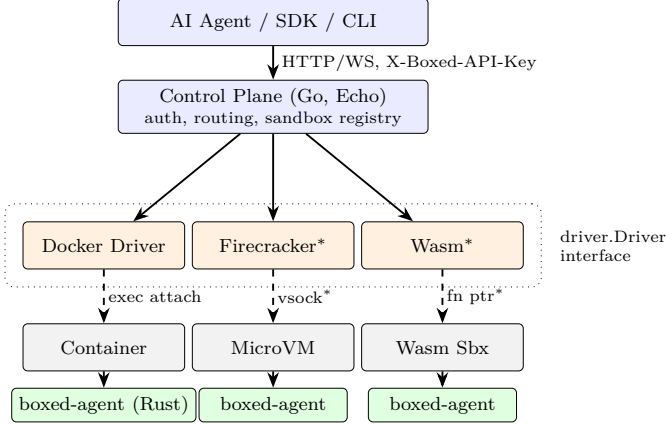


Fig. 2. Boxed architecture. The control plane drives a single narrow Driver interface, behind which the Docker driver ships today and Firecracker/Wasm drivers are stubbed. A common Rust agent runs inside every sandbox and speaks JSON-RPC over whatever byte stream the driver supplies. *Not implemented in the current release.

```
}
}
```

Create instantiates a sandbox from the supplied configuration and returns an identifier; Start and Stop bound its lifetime; Connect returns a raw byte stream over which the control plane talks to the in-sandbox agent. For the Docker driver that stream is a Docker exec attach; for the planned Firecracker driver it will be a vsock connection; for a future Wasm driver it will be a host-side function pointer. The control plane is agnostic to which case it is servicing, which is what makes backend selection a deployment-time rather than a compile-time decision.

B. Control Plane

The control plane is a stateless HTTP and WebSocket server built on the Echo framework that maps POST /v1/sandbox requests through the configured driver. Two choices matter for agent workloads.

First, create returns once the underlying runtime reports that the sandbox process has launched (for the Docker driver, once ContainerStart acknowledges) with status: ready. The caller does not poll a separate status endpoint. The first exec after create absorbs the small additional cost of waiting for the agent’s JSON-RPC loop

to come up. Because an agent task may create dozens of short-lived sandboxes, removing a polling round-trip from each one is a material saving in the aggregate.

Second, authentication uses an operator-configured static API-key header (X-Boxed-API-Key). Boxed does not issue, rotate, or store keys. This is deliberately a single-instance authentication mechanism, not a multi-tenant identity, key-management, or compliance system. In a self-hosted deployment, the operator controls the infrastructure that receives generated code, execution output, and intermediate files.

C. In-Sandbox Agent

Inside every sandbox runs boxed-agent, a single statically linked Rust binary built on tokio, serde, and notify (1.32 MiB stripped). It speaks JSON-RPC 2.0 framed as newline-delimited JSON over the driver-supplied byte stream.

On exec, the agent spawns the user’s command as a child process and streams stdout and stderr chunks back as discrete RPC events as they appear, rather than buffering until exit. This streaming discipline matters for agent workloads in which the model must react to partial output or an early error signal without waiting for the program to finish. In parallel, an inotify-based watcher monitors the artifact directory (/output); any file written there is emitted as an artifact event carrying the file’s MIME type and base64-encoded contents. An agent that asks Python to write plot.png therefore receives the bytes as an unsolicited event, with no second round-trip and no polling.

D. Artifact Protocol

Treating files as first-class outputs is what makes the substrate practical for workloads where the model expects the side effects of code, not only its console output. Fig. 3 shows where artifact events sit in the sandbox lifecycle.

The current release implements inline streaming: the watcher reads the file, guesses its MIME type, base64-encodes it, and emits it on the same channel that carries stdout and stderr. A size cap of 10 MiB bounds the memory and serialization cost of inlining; files above the cap are currently skipped with a logged warning rather than delivered. The wire protocol reserves a url field on the

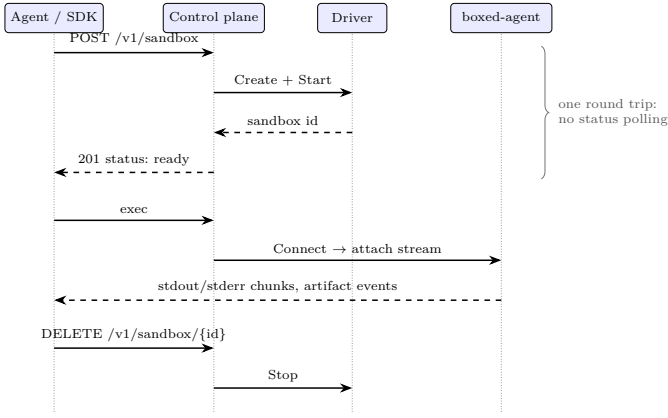


Fig. 3. Sandbox lifecycle. create returns as soon as the runtime reports the sandbox process launched, so the client never polls a status endpoint; the first exec absorbs agent start-up. Output and artifacts stream back on one channel as they are produced.

artifact event for an out-of-band upload path, in which the agent would PUT the file to a pre-signed URL and return only a reference, but that path is not implemented in this release. This is flagged explicitly because a substrate that silently drops large artifacts is a correctness hazard for any workload that emits model checkpoints or datasets; closing this gap is scheduled ahead of the Firecracker driver.

Because both the artifact protocol and the agent live above the driver interface, artifact semantics are invariant across backends: the Docker driver today and the Firecracker and Wasm drivers later expose identical behaviour to the caller.

V. Implementation

The current release comprises 2,374 lines of Go in cmd and internal (control plane, Docker driver, CLI) and 870 lines of Rust for the in-sandbox agent. The control-plane binary is 12.0 MiB and the agent binary 1.32 MiB stripped. Three implementation details are worth stating precisely, because two of them differ from what a reader might assume.

A. Agent injection and container lifecycle

The Docker driver bind-mounts the host’s boxed-agent binary into the container read-only at create time, so a stock base image (python:3.10-slim) can be reused across sandboxes with no Boxed-specific image registry to maintain. The image root filesystem is read-only. Separate tmpfs mounts back /tmp, /output, and the configured working directory, so user-written data is ephemeral and never touches the host filesystem.

The container’s own PID 1 is a tail -f /dev/null placeholder whose only job is to keep the namespace alive; boxed-agent is launched separately by Connect as a Docker exec whose attached stream carries JSON-RPC. The distinction is stated explicitly because it has security consequences: the agent is a child of the container’s init rather than init itself, so a workload sharing the

sandbox’s PID namespace can see it, which is one reason the permissive ptrace default discussed in Section VI matters. Under the planned Firecracker driver the agent does become the guest init, communicating over vsock; the interface in Section IV-A is what allows that change without disturbing the control plane.

B. Single-round creation

POST /v1/sandbox returns once the runtime has launched the sandbox process rather than requiring the caller to poll a status endpoint (Fig. 3). The first exec after create absorbs the latency of the agent’s JSON-RPC loop initializing. The client therefore observes a simple request–response model, and per-sandbox polling load disappears from the control plane, which matters precisely because agent tasks create sandboxes in bulk.

C. Resource and network model

Every sandbox is created with a CPU quota (NanoCPUs), a memory cgroup, and a PidsLimit of 256. The container root is read-only, all Linux capabilities are dropped, and no-new-privileges is enabled. The resource defaults remain 1.0 core and 512 MiB, with ceilings of 4.0 cores and 8 GiB. A TTL goroutine stops the sandbox when its timeout expires.

The Docker backend always uses Docker’s none network, leaving only loopback. Requests that enable networking or specify an allow-list are rejected until Boxed implements an enforceable per-sandbox egress policy. This is a deliberate fail-closed restriction rather than a claim that Docker’s default bridge is a sufficient egress control.

VI. Evaluation

A. Evaluation status

The CSV traces included with this revision were collected before the hardening changes described in Section V and before a defect in the agent’s exit-status reporting was corrected. The agent emitted exit code zero after a command stream ended, so the historical pass/fail labels in the agent trace do not establish task correctness. Likewise, the escape-probe table describes the pre-hardening Docker configuration, not the current implementation. These results are retained as an auditable baseline and are not evidence for the revised implementation. Before submission, the full suite must be rerun on the hardened revision, on repeated trials, and on a native Linux host.

The evaluation covers five axes: cold-start latency, throughput under concurrency, idle per-sandbox overhead, adversarial escape resistance, and end-to-end LLM-agent task latency. The harness and raw CSV traces are open-source alongside the paper.

B. Setup

All measurements were taken on a MacBook Pro with an Apple M1 Pro SoC and 16 GB of unified memory running macOS, using Docker Desktop with the arm64

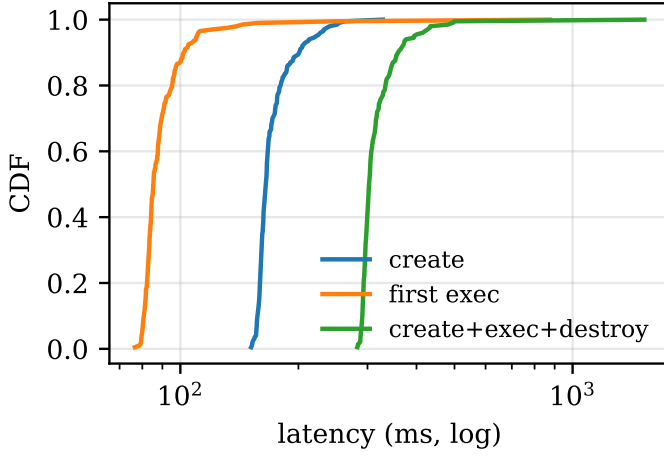


Fig. 4. CDF of create, first-exec, and total latency over 200 sequential sandboxes.

python:3.10-slim image pre-pulled to keep image-pull latency out of the measured path. Cold-start, throughput, and overhead numbers were collected on a freshly restarted Docker engine. The agent-trace experiment was deliberately run after the three load-generating phases, to characterize behaviour under the accumulated state of a Docker Desktop hypervisor that has been under sustained pressure. Unless otherwise noted each create+exec+destroy cycle runs a synthetic print('ok') workload, which isolates substrate overhead from application computation. Only the Docker driver is exercised; Firecracker and Wasm remain stubs.

C. Cold-Start Latency

End-to-end create+exec+destroy time was measured over 200 sequential sandboxes on a warm engine (Fig. 4). The median is 303ms with a tight interquartile band (IQR 28ms) and a heavy upper tail (p95 395ms, p99 495ms). The create phase alone has a median of 165ms, making it the dominant contributor. The bulk of that is container-instantiation cost inside the Docker driver; the agent adds single-digit milliseconds between create returning and the first exec acknowledgement. This is a single benchmark run on a single host; we release the harness so that replication across hardware and operating systems is possible, and treat that replication as future work rather than a claim we have discharged.

D. Throughput Under Concurrency

Client concurrency was swept over $c \in \{1, 2, 4, 8, 16, 32\}$, driving 80 sandbox lifecycles at each level, and report sustained sandboxes per second (Fig. 5, Table II). Throughput rises monotonically to 9.8 sandboxes/s at $c=8$, dips slightly at $c=16$, and collapses to 2.35 sandboxes/s at $c=32$, where elapsed time for the same 80 lifecycles grows roughly $4\times$. No errors occurred at any level. The $c=16$ dip is read as host-noise variance rather than a structural feature and do not over-interpret the curve shape; the

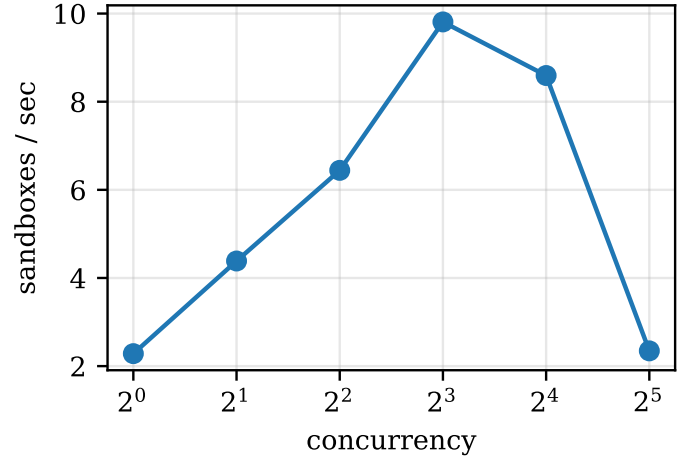


Fig. 5. Sustained sandbox-creation rate as a function of client concurrency.

TABLE II
Throughput sweep. 80 sandbox lifecycles per concurrency level, single trial each, zero errors throughout.

Concurrency c	Completed	Elapsed (s)	Sandboxes/s
1	80	35.04	2.28
2	80	18.24	4.39
4	80	12.42	6.44
8	80	8.16	9.81
16	80	9.31	8.59
32	80	34.11	2.35

$c=32$ collapse is large enough to be a real contention effect in Docker Desktop’s LinuxKit VM, which multiplexes every container operation through a single virtualized kernel. Section VII discusses why we expect a native Linux/Firecracker deployment to lift this ceiling.

E. Per-Sandbox Idle Overhead

Each idle sandbox’s resident-set size and CPU were sampled five seconds after create returns (Fig. 6). Median RSS, as reported by docker stats via the container cgroup working set, is 0.4 MiB; median CPU is 0.02%.

Two caveats apply. On macOS, Docker Desktop runs containers inside a LinuxKit VM, so the working-set figure reflects accounting inside that VM rather than a process-level measurement on a native Linux host, and may under-report. We therefore treat the absolute number as a relative ordering rather than ground truth and plan re-measurement on Linux. That said, the order of magnitude is consistent with the design: the agent is a stripped Rust binary that holds no buffers, opens no files at startup, and parks on a single tokio event loop awaiting a JSON-RPC frame, in contrast to a Python- or Node-based agent carrying an interpreter, module cache, and garbage-collector metadata even when idle. Across the dozens of concurrent sandboxes an agent operator may run, that difference compounds.

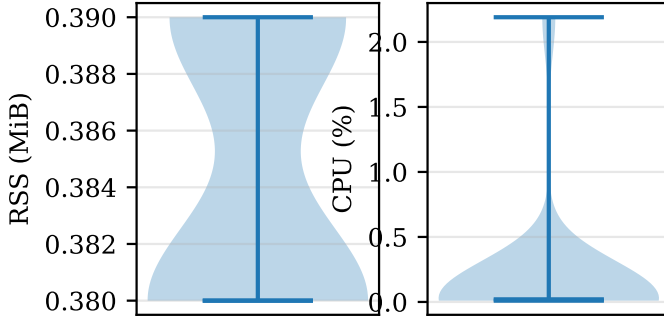


Fig. 6. Idle resident-set size and CPU per sandbox.

TABLE III

Behavioural escape probe: all twelve vectors, mapped to the threat classes of Section III, measured before the hardening revision. 5/12 (42%) denied. “Locus” records where the denial or failure plausibly originates; entries marked † are classifier artifacts discussed in the text, not demonstrated boundary crossings.

Vector	Threat	Verdict	Locus / gap exposed
mount_host	T1,T2	denied	no CAP_SYS_ADMIN
modprobe	T2	denied	no CAP_SYS_MODULE
setns	T2	denied	namespace caps absent
rfc1918	T3	denied	Docker NAT, not Boxed policy
imds	T3	denied	Docker NAT, not Boxed policy
ro_root	T1	undecided	ReadonlyRootfs unset
raw_socket	T2	undecided	CapDrop: ALL disabled
ptrace_host	T2,T3	undecided	seccomp permits ptrace
fork_bomb	T4	undecided	no PidsLimit cgroup
mem_bomb	T4	undecided	† signature not emitted
proc1_root	T1	undecided	† in-sandbox root only
docker_sock	T2,T3	undecided	† socket not mounted

F. Pre-hardening Behavioural Escape Probe

The sandbox was exercised with twelve attempts drawn from common container-escape patterns [3]: privileged mount, `/proc/1/root` traversal, Docker socket access, kernel module loading, raw-socket binding, namespace escape via `unshare`, egress to RFC1918, egress to link-local metadata (169.254.169.254), a fork bomb, a 4 GiB memory bomb, a write to `/etc`, and `ptrace` of a non-self process. Each attempt runs in a fresh sandbox and is classified by matching captured `stdout/stderr` against a list of denial signatures. Table III gives the full per-vector result.

This is a behavioural probe, not a formal isolation proof, and we are careful about what the numbers license. The classifier cannot distinguish an attempt denied by Boxed policy from one denied incidentally by the Docker bridge or kernel defaults: the RFC1918 and metadata-endpoint denials are plausibly the work of Docker Desktop’s NAT topology rather than any enforcement of ours, and we record them as such in Table III. With that caveat, the suite classifies 5 of 12 attempts (42%) as denied.

Four of the seven undecided vectors correspond to configuration gaps in the pre-hardening Docker driver: (i) the container rootfs is writable because `ReadonlyRootfs` is never set, so the write to `/etc` lands; (ii) the `CapDrop: ALL` block is commented out, so the container retains Docker’s

default capability set including `CAP_NET_RAW`, and the raw-socket bind succeeds; (iii) no `PidsLimit` cgroup is configured, so the fork bomb is bounded only by whatever the host scheduler tolerates, even though a memory cgroup is enforced; and (iv) the default seccomp profile permits `ptrace` within the sandbox PID namespace, which is a sharper concern here than it would be otherwise because Section V establishes that boxed-agent is a peer process in that namespace rather than its init.

The remaining three undecided verdicts are, on inspection, artifacts of signature-based classification rather than evidence of a failed boundary, and we mark them † rather than quietly counting them as successes for the adversary. The memory bomb requests a 4 GiB allocation against a 512 MiB cgroup; a process terminated by `SIGKILL` emits no message, so no denial signature appears in captured output even when the cgroup binds exactly as intended. `/proc/1/root` resolves to the root of the container’s own init inside the sandbox PID namespace, not the host’s, so listing it succeeds without crossing any boundary. And the Docker socket is simply not among the mounts the driver configures, so the probe observes an absent path. In each case the honest reading is that our classifier is too weak to adjudicate the vector, not that the sandbox failed. Replacing signature matching with post-condition verification against host state, did the file actually appear outside the mount namespace? did the process count actually exceed the cap?, would remove this ambiguity and is the natural next iteration of the suite.

The aggregate is reported as 5/12 rather than adjusting it upward for the three artifacts, because the classifier is what we actually ran and re-scoring after the fact on the basis of a plausible reading would be exactly the kind of favourable post-hoc adjustment that makes security evaluations untrustworthy. None of the twelve attempts produced a host escape in our tests. But each genuine gap thins the defence chain an operator would reasonably expect, which is why we treat them as bugs against this release rather than as design limitations.

G. Pre-fix Agent Trace

Boxed was driven by a Gemini-based code-generation loop over 20 HumanEval-style tasks [18]: for each task the model emits a candidate Python function, Boxed executes it together with a self-checking assertion, and the harness records pass/fail. The historical report that all 20 tasks passed is invalid because the pre-fix agent emitted a zero exit status regardless of command outcome. Figure 7 and the associated timing trace remain useful only as a pre-fix operational trace. The reported median end-to-end wall-clock was 30.32 s, of which 17% is model inference and the remainder is dominated by sandbox creation.

The striking number is the create phase: under sustained load it degraded to a median of 15.32 s, roughly two orders of magnitude above the 165 ms median measured on a freshly warmed engine. Because this phase is the same

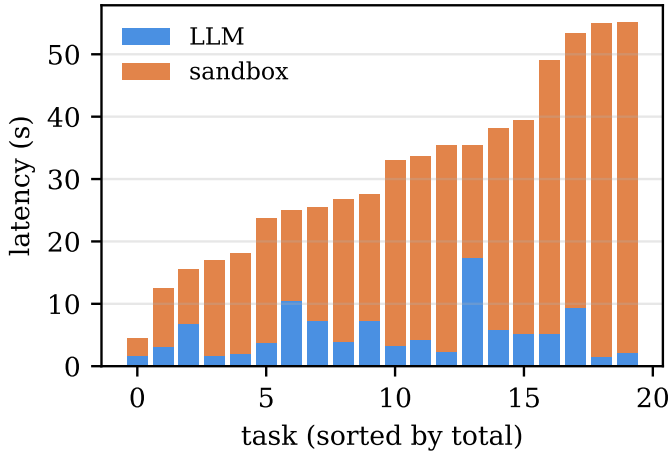


Fig. 7. Per-task LLM time versus sandbox time over 20 HumanEval-style code-generation tasks.

code path in both experiments and the only variable is the accumulated state of Docker Desktop’s LinuxKit VM after three prior load-generating phases, we attribute the degradation to hypervisor contention rather than to the Boxed runtime, but we note it is an attribution supported by the experimental ordering, not by a controlled comparison, and a Linux replication would settle it. Either way the gap motivates both native Linux with Firecracker as a deployment target and pool-based reuse as control-plane work (Section VII).

H. Threats to Validity

All measurements were taken on a single MacBook Pro (M1 Pro, 16GB) running typical background processes (browser, IDE, chat clients) rather than a quiesced benchmark host. We did not pin Boxed’s processes to dedicated cores, isolate Docker Desktop’s LinuxKit VM, or otherwise insulate the workload from incidental host activity. The heavy upper tail of the cold-start distribution (p99 495ms versus median 303ms) and the dip at $c=16$ are both consistent with sporadic host CPU contention rather than steady-state behaviour of Boxed.

Reporting median and IQR partially insulates central tendency from that contention, but the absolute numbers should be read as upper bounds on a contended developer environment, not as clean-room results. The throughput sweep is a single trial per concurrency level, which precludes any variance estimate or confidence interval. The escape probe is a behavioural signature matcher whose limits we quantify in Section VI-F. The comparison figures in Table I for other systems are vendor claims we did not reproduce. We release the full harness, including every caveat above, so that independent re-runs on quiesced Linux hosts can refine the picture.

TABLE IV
Hardening checklist for the current Docker driver.

Historical gap	Vector	Current status
Writable rootfs	ro_root	fixed: ReadonlyRootfs
Default capability set	raw_socket	fixed: CapDrop: ALL
No process cap	fork_bomb	fixed: PidsLimit
Permissive ptrace	ptrace_host	pending: custom seccomp profile
No egress enforcement	,	fail closed: networking rejected
Oversize artifacts dropped	,	implement upload path

VII. Discussion

A. Hardening status

The Docker driver is the only implemented backend. The root filesystem, capability, process-count, privilege-escalation, and default-networking gaps identified by the historical probe have been fixed in the current source. Table IV records the corresponding changes. A custom seccomp profile and fine-grained egress allow-list remain unimplemented; networked execution is therefore rejected. The revised driver still requires a fresh, post-condition-based security evaluation before it can support a security claim.

B. Docker Desktop on macOS is a hypervisor

Our harness ran on Apple M1 Pro under Docker Desktop, which is itself a Linux VM. The cold-start numbers are fast in absolute terms but include the cost of round-tripping through the Docker socket inside that VM, and the throughput collapse at $c=32$ reflects contention for CPU and I/O inside a fixed-size LinuxKit VM that must multiplex every container operation through one virtualized kernel. We expect a native Linux deployment with a Firecracker driver to reduce create latency substantially and to raise the concurrency ceiling, because sandbox creation would distribute across the full physical core count without the hypervisor hop. This is stated as an expectation, not a result: we have not measured it, and the Firecracker driver is a stub.

C. Pool-based sandbox reuse

The control plane maintains a logical sandbox pool, but the Docker driver creates a fresh container per request. That choice buys a known-clean starting state, no residual filesystem or process state from a prior execution, at the cost of paying 165ms of container instantiation on every call, which Section VI identifies as the dominant term in the 303ms median.

Reusing a warmed sandbox for compatible follow-up requests would amortize that cost. Oakes et al.’s SOCK [19] demonstrates the approach in a serverless setting, using an LRU cache of paused containers to avoid repeated instantiation, which is evidence that the design works even

though their access pattern differs from an agent’s. Compatibility is naturally keyed on the tuple of base image, network policy, and resource limits; a reset between uses (killing descendant processes, clearing /tmp and /output, dropping injected files) is far cheaper than a full teardown and recreate. The roadmap has three parts: an LRU eviction policy bounding idle sandboxes, a compatibility-keyed lookup from SandboxConfig to pool entries, and a reset handshake the in-sandbox agent acknowledges over JSON-RPC. The driver interface already accommodates this: Create may return an existing identifier and Connect opens a fresh RPC channel to the same agent, so neither the SDK nor the client-facing API changes. For an agent workload that spawns dozens of sandboxes per task, this is the single highest-leverage optimization remaining.

D. Multi-host scheduling

Boxed today is single-host. A scheduler placing sandboxes across worker nodes, and accounting for NetworkPolicy at placement time so that co-located sandboxes cannot hold conflicting egress policies, is the natural next step for deployments whose aggregate concurrency exceeds one machine.

E. A Wasm driver, when the ecosystem allows

Wasm/WASI [16] is the most interesting future backend because of microsecond-scale instantiation and capability-based isolation enforced at the bytecode level, which would eliminate kernel boot and namespace setup from the create path entirely. The blocker is workload compatibility: agents emit Python scripts, shell commands, and native binaries that assume POSIX filesystem access, process spawning, and dynamic linking, none of which WASI transparently provides today. Adding the driver when the toolchain matures means implementing four methods and mapping Connect to an in-Wasm agent compiled for WASI; the control plane, artifact protocol, and SDK are unchanged. That is the payoff of keeping the interface narrow.

VIII. Future Work

Four directions follow directly from the limitations reported above.

Validating the hardened implementation. The root filesystem, capability, process-count, privilege-escalation, and default-networking changes have been applied in the source. The immediate priority is to rerun the twelve-vector probe using post-condition checks and report the revised per-vector table, so that the effect of each change is attributable.

Completing the Firecracker driver. The driver interface was designed so that a stronger isolation backend can be substituted without altering the control plane, and the Firecracker backend is currently a stub. Completing it would place a virtualization boundary between the sandbox and the host, which addresses the escape vectors

that remain open under the shared-kernel Docker driver, and would test whether the four-method interface is in fact sufficient for a microVM.

Reducing per-execution cost through sandbox reuse. Cold start dominates the measured latency, and the current design pays it on every task. A pool of warm sandboxes would amortize creation across executions, at the cost of weakening the guarantee that each execution begins from a clean state. The trade-off between reuse depth and isolation strength is worth measuring rather than assuming.

Evaluation beyond a single host. All measurements here come from one laptop under Docker Desktop, which on macOS interposes a hypervisor. A native Linux deployment and a multi-host scheduler would establish how much of the observed latency is attributable to that layer, and would extend the throughput results past the point where a single machine saturates.

IX. Conclusion

Boxed is a Docker-based execution substrate for an LLM agent that creates, exercises, and destroys sandboxed environments many times per task. It provides a streaming in-sandbox agent and an interface intended to accommodate future backends. The present implementation defaults to a restricted Docker configuration and fails closed on networking until policy enforcement exists.

The historical performance, security, and agent-trace data are retained for auditability, but do not support claims about this revised implementation. In particular, the agent pass/fail labels were invalidated by an exit-status bug; the historical escape probe predates hardening; and the performance data is a single-run Docker Desktop measurement. Repeated native-Linux measurements, post-condition-based security tests, verified agent-task correctness, and an implemented second backend are required before this manuscript is submitted as an IEEE research paper.

Availability

The complete implementation, Go control plane, Rust in-sandbox agent, Docker driver, benchmark harness, raw CSV traces, and reproduction instructions, is available under the MIT license at <https://github.com/akshayaggarwal99/boxed>. Every performance number in this paper is generated from those traces by `bench/analyze/stats.py` and injected into the manuscript as a macro, so the text cannot drift from the data.

References

- [1] J. Yang, C. E. Jimenez, A. Wettig et al., “SWE-agent: Agent-computer interfaces enable automated software engineering,” in NeurIPS, 2024.
- [2] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, “Executable code actions elicit better LLM agents,” in International Conference on Machine Learning (ICML), 2024.
- [3] NIST NVD, “CVE-2019-5736: runc container breakout,” <https://nvd.nist.gov/vuln/detail/CVE-2019-5736>, 2019.

- [4] X. Lin, L. Lei, Y. Wang, J. Jing, K. Sun, and Q. Zhou, “A measurement study on Linux container security: Attacks and countermeasures,” in Annual Computer Security Applications Conference (ACSAC), 2018.
- [5] X. Gao, Z. Gu, Z. Li, H. Jamjoom, and C. Wang, “Houdini’s escape: Breaking the resource rein of Linux control groups,” in ACM SIGSAC Conference on Computer and Communications Security (CCS), 2019.
- [6] E2B, “E2B: Open-source secure sandboxes for AI agents,” <https://e2b.dev>, 2024.
- [7] Modal Labs, “Modal Sandbox documentation,” <https://modal.com/docs/guide/sandbox>, 2025.
- [8] Cloudflare, “Cloudflare Sandbox SDK,” <https://developers.cloudflare.com/workers/sandbox/>, 2025.
- [9] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, “Firecracker: Lightweight virtualization for serverless applications,” in USENIX NSDI, 2020.
- [10] Google, “gVisor: Container runtime sandbox,” <https://gvisor.dev>, 2018.
- [11] E. G. Young, P. Zhu, T. Caraza-Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “The true cost of containing: A gVisor case study,” in USENIX Workshop on Hot Topics in Cloud Computing (HotCloud). USENIX Association, 2019.
- [12] Kata Containers Project, “Kata Containers: Secure containers with the speed of containers,” Open Infrastructure Foundation project, <https://katacontainers.io>, 2018.
- [13] F. Manco, C. Lupu, F. Schmidt, J. Mendes, S. Kuenzer, S. Sati, K. Yasukata, C. Raiciu, and F. Huici, “My VM is lighter (and safer) than your container,” in SOSP, 2017.
- [14] A. Madhavapeddy, R. Mortier, C. Rotsos, D. Scott et al., “Unikernels: Library operating systems for the cloud,” in ASPLOS, 2013.
- [15] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. Bastien, “Bringing the web up to speed with WebAssembly,” in ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2017.
- [16] Bytecode Alliance, “WebAssembly system interface (WASI),” <https://wasi.dev>, 2024.
- [17] Daytona, “Daytona: Open-source dev-environment platform,” <https://daytona.io>, 2024.
- [18] M. Chen, J. Tworek, H. Jun et al., “Evaluating large language models trained on code,” arXiv preprint arXiv:2107.03374, 2021.
- [19] E. Oakes, L. Yang, D. Zhou, K. Houck, T. Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “SOCK: Rapid task provisioning with Serverless-Optimized containers,” in USENIX Annual Technical Conference (USENIX ATC). USENIX Association, 2018, pp. 57–70.